

SIMULADO A1

Total: 60 pontos

Instruções gerais

- Leia cada questão com atenção. Códigos-base já estão fornecidos — **você só precisa escrever os testes.**
- Entregue: código dos testes (JUnit/Mockito, FitNesse pages/fixtures, Bruno collections, JMeter .jmx) + passo a passo de execução no `README.md`.
- Entregue um ZIP com todo o diretório/WorkSpace. Nome do arquivo sugerido: `SimuladoA1_<seunome>.zip`.

Estrutura de pastas (obrigatória)

```
SimuladoA1/
├── 1-JUnit/
│   ├── pom.xml
│   └── src/
│       ├── main/java//a1/    (códigos-base Java)
│       └── test/java//a1/    (seus testes JUnit/Mockito)
├── 2-FitNesse/
│   ├── FitNesseRoot/        (páginas wiki e fixtures)
│   └── src/                   (código Java de fixtures se necessário)
├── 3-Postman/
│   ├── api-node/             (API NodeJS fornecida)
│   ├── api-python/           (API Python fornecida)
│   └── collections/           (sua collection .json)
├── 4-JMeter/
│   ├── testes/               (seu arquivo .jmx)
│   └── README-JMeter.md
└── README.md                 (instruções gerais para executar)
```

Tema 1 — Teste Unitário e Mock com JUnit (30 pts / 8 questões)

Tecnologia: Java (JUnit 5 + Mockito). Projeto Maven (pom.xml já configurado).

Código-base (disponível em `1-JUnit/src/main/a1/`)-

Questões JUnit (8 questões = 30 pts)

Q1 — Teste de soma (3 pts — Fácil)

- Crie `CalculadoraTest` e escreva um teste para `somar(2,3) == 5`.

Q2 — Teste de subtração (3 pts — Fácil)

- Escreva teste para `subtrair(10,4) == 6`.

Q3 — Teste de multiplicação (3 pts — Fácil)

- Teste `multiplicar(6,7) == 42`.

Q4 — Divisão e exceção (4 pts — Média)

- Verifique `dividir(10,2) == 5`.
- Verifique que `dividir(10,0)` lança `ArithmeticException` com `assertThrows`.

Q5 — Teste parametrizado (4 pts — Média)

- Crie um `@ParameterizedTest` para `somar` com os casos: (1,1 → 2), (3,7 → 10), (-5,5 → 0).

Q6 — Mockito: mock simples (5 pts — Média/Avançado)

- Escreva teste para `PedidoService` onde `EmailService` é **mockado** retornando `true`.
- Asserte que `finalizarPedido("c@e.com", "SKU-1", 1)` retorna `"sucesso"`.
- Verifique `emailService.enviarEmail(...)` foi chamado **1 vez** com Mockito `verify()`.

Q7 — Mockito: comportamento de falha (4 pts — Média)

- Mock `EmailService` retornando `false`. Teste que `finalizarPedido(...)` retorna `"falhou"`.

Q8 — Mockando dependência de estoque (4 pts — Avançado)

- Use Mockito para mockar `EstoqueService` e testar o caminho `"sem_estoque"` quando `verificarEstoque` retorna `false`. Asserte que `finalizarPedido` retorna `"sem_estoque"` e **não** chama `emailService.enviarEmail()` (use `verify(emailService, never())`).

Entregáveis para Tema 1

- Todos os arquivos de teste em `1-JUnit/src/test/java/a1/`.
- Um `pom.xml` mínimo com JUnit5 e Mockito (instruções no README).

Tema 2 — Teste Funcional e Aceitação com FitNesse (10 pts / 3 questões)

Tecnologia: FitNesse + fixtures Java. Rodar FitNesse localmente (jar).

Código-base e instruções

- Pasta: `2-FitNesse/FitNesseRoot/`
- Forneça páginas wiki (arquivos `.wiki` ou estrutura FitNesse) com cenários e instruções.
- Fixtures podem usar as mesmas classes Java (ex.: `Calculadora`) — implemente fixtures simples.
- Coloque os códigos abaixo nas páginas para encontrar as classe

`!path ./target/classes`

`!path ../1-JUnit/target/classes`

Código-base fixture (`2-FitNesse/src/`) Questões FitNesse (3 questões = 10 pts)

Q9 — FitNesse: Teste funcional de soma (3 pts — Fácil)

- Crie uma página FitNesse `Calculadora.Somar` que use `CalculadoraFixture` para testar soma de 2 conjuntos de dados.

Q10 — FitNesse: Cenário de aceitação (4 pts — Média)

- Crie página `Pedido.FinalizarPedido` que usa fixtures para simular chamada à `PedidoService`.
- Cenário 1: estoque ok → espera `sucesso`.
- Cenário 2: sem estoque → espera `sem_estoque`.

Q11 — FitNesse: Teste de integração (3 pts — Média)

- Crie uma suíte com página `SuiteSimuladoA1` que reúne `Calculadora.Somar` e `Pedido.FinalizarPedido`.
- Instruções de execução: como gerar relatório e capturar screenshot/texto de output.

Entregáveis para Tema 2

- Diretório `2-FitNesse/` com páginas, fixtures source, e README com comando para rodar FitNesse (ex.: `java -jar fitnesse-standalone.jar -p 8082` e abrir `http://`

localhost:8082/).

Tema 3 — Teste de Serviços Web com Bruno (10 pts / 3 questões)

Tecnologia: Postman collection (.json). Testar as APIs NodeJS e Python fornecidas.

API NodeJS fornecida (3-Postman/api-node/)

API Python fornecida (3-Postman/api-python/)

Questões Bruno (3 questões = 10 pts)

Q12 — Teste REST básico (3 pts — Fácil)

- Crie uma request `GET /api/status` (NodeJS) e escreva teste no Bruno que valida `status == "ok"`.

Q13 — Teste de contrato e checagem de erro (4 pts — Média)

- Crie request `POST /api/orders` com body inválido (sem `product`) e valide que o status é `400` e campo `error` existe.
- Crie request válido e valide `201` + esquema básico (tem `id` e `product`).

Q14 — Collection com ambientes (3 pts — Média)

- Organize as requests do Postman em uma **collection** com baseUrl em `http://localhost:5000`
 - Criar requests na collection usando variáveis de ambiente `{{baseUrl}}` para apontar para a API correta.
 - Requests incluídas:
 - `GET /api/status` (NodeJS)
 - `POST /api/orders` (NodeJS)
 - `GET /api/ping` (Python)
 - `POST /api/process` (Python)
 - Criar **tests** em cada request para validar respostas JSON esperadas:
 - Status HTTP correto (200 ou 201)

- Campos retornados (`status` , `id` , `product` , `pong` , `input`)
- Payload correto ao enviar dados
- Exportar a collection como arquivo `.json` e entregar junto com os ambientes `.json`.

Entregáveis – Tema 3: Teste de Serviços Web com Bruno (10 pts / 3 questões)

Para cada questão, o aluno deverá entregar os seguintes itens:

1. Collection Insomnia (.json)

- Uma collection que inclua todas as requests solicitadas:
 - `GET /api/status` (NodeJS)
 - `POST /api/orders` (NodeJS)
 - `GET /api/ping` (Python)
 - `POST /api/process` (Python)
- Todas as requests devem utilizar **variáveis de ambiente** (`{{baseUrl}}`) para permitir alternar facilmente entre NodeJS (`http://localhost:3000`) e Python (`http://localhost:5000`).

2. Testes automatizados em cada request

- Cada request deve incluir **scripts de teste** no Postman para validar:
 - Código de status HTTP correto (`200` ou `201`)
 - Presença e valores esperados de campos no JSON (`status` , `id` , `pong` , `input`)
 - Erros esperados para payloads inválidos (ex.: `400` com campo `error`)

Tema 4 — Teste de Performance com JMeter (10 pts / 4 questões)

Tecnologia: JMeter (.jmx) apontando para a API Python (fornecida). Fornecer arquivo `.jmx` e instruções.

Utilize as APIs do Python ou NodeJS

Questões JMeter (3 questões = 10 pts)

Q15 — Plano de carga (4 pts — Média)

- Criar `.jmx` que execute `GET /api/ping` com 100 threads, duração 60s.
- Validar que 95% das respostas tenham tempo < 200ms (documente como obter o relatório).

Q16 — Teste de estresse (3 pts — Média/Avançado)

- Criar `.jmx` que execute `POST /api/orders` para fazer rampa até 500 threads em 2 minutos, mantendo para 1 min; medir taxa de erro.

Q17 — Teste de cenário negócios (3 pts — Média)

- Criar thread group que faz `POST /api/process` com payload JSON (ex.: `{"orderId": "123", "qty": 2}`) com 50 threads por 60s.

Entregáveis para Tema 4

- `4-JMeter/testes/simuladoA1.jmx` com os Thread Groups
- `README-JMeter.md` com instruções: como abrir no JMeter GUI e gerar relatório HTML (`jmeter -n -t ... -l ... -e -o report/`).